

AnyBank Agentic Workflows for Intelligent Mortgage Processing

Stakeholder

Name	Role	email
Chief Information Officer	Jeejee Unwalla	ujeejee@amazon.com
Chief Technology Officer	Ali Ghomi	alighomi@amazon.com
Head of Business Unit	Prasad Gunaratne	prgun@amazon.com
Chief Risk Officer	Paul Curtis	paucurt@amazon.com
Security Architect	Prasad Gunaratne	prgun@amazon.com

Overview

This repository provides an automated loan application processing system that streamlines document submission, verification, underwriting and approval processes. The solution helps financial institutions reduce manual processing time, minimize errors, and provide better customer experience through AI-powered document processing and validation.

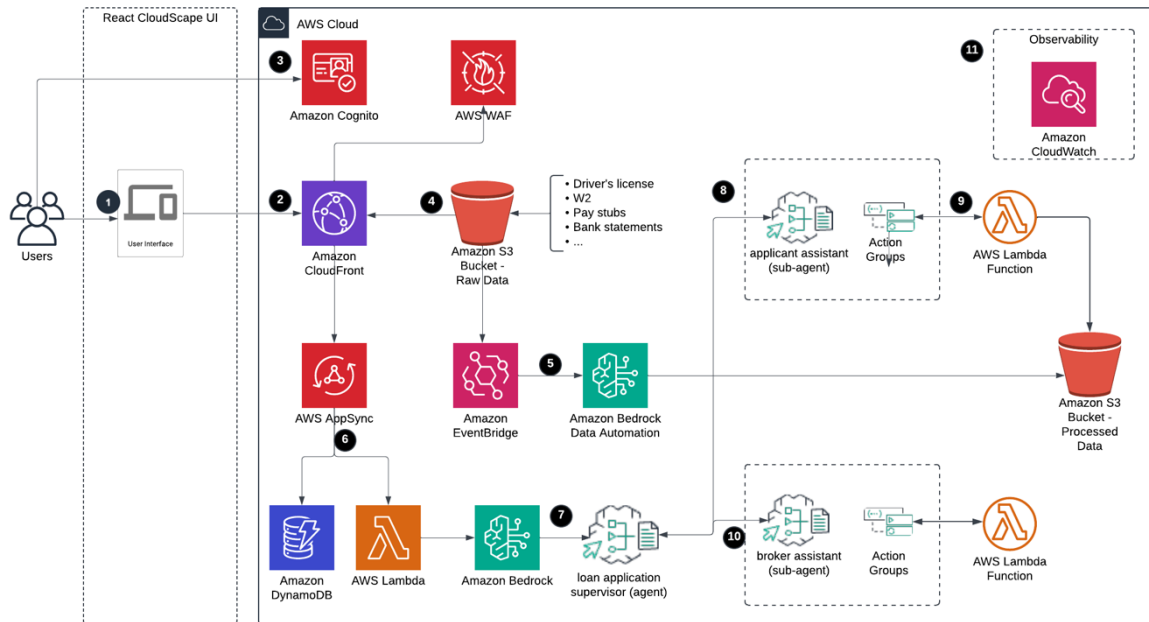
Table of Contents

- [Overview](#)
- [Key Features](#)
- [Architecture](#)
 - [Architecture Diagram](#)
 - [Provisioned Resources](#)
- [Demo Script](#)
- [Deployment Steps](#)
 - [Prerequisites](#)
 - [Quick Start](#)
- [Cost Estimation](#)
 - [Cost Breakdown](#)
- [FAQ](#)

Key Features

- Intelligent document upload and processing (PDF/Images)
- Automated data extraction from key documents
- Real-time document validation and verification
- Automated loan application form completion
- DTI calculation and pre-approval letter generation
- Interactive chatbot for loan process assistance

Architecture



Network Architecture Design

The network architecture is built on a secure VPC foundation with proper segmentation across multiple availability zones.

VPC Foundation

Component	Details
CIDR	10.0.0.0/16
Regions	Primary (us-west-2) + DR region
AZ-1	10.0.1.0/24 (Public), 10.0.11.0/24 (Private), 10.0.21.0/24 (Data)
AZ-2	10.0.2.0/24 (Public), 10.0.12.0/24 (Private), 10.0.22.0/24 (Data)
AZ-3	10.0.3.0/24 (Public), 10.0.13.0/24 (Private), 10.0.23.0/24 (Data)

Network Segmentation

Tier	Components	Subnets
Tier 1 - Web/Presentation	CloudFront Distribution, Application Load Balancer, WAF	Public Subnets (10.0.1.0/24, 10.0.2.0/24, 10.0.3.0/24)
Tier 2 - Application	Lambda Functions, API Gateway, Cognito User Pool	Private Subnets (10.0.11.0/24, 10.0.12.0/24, 10.0.13.0/24)
Tier 3 - Data/AI	Amazon Bedrock, S3 Buckets, EventBridge	Data Subnets (10.0.21.0/24, 10.0.22.0/24, 10.0.23.0/24)

Security Boundaries

Security Groups Configuration

Group	Inbound Rules	Outbound Rules
Web-Tier-SG	Port 443 (HTTPS) from 0.0.0.0/0; Port 80 (HTTP) from 0.0.0.0/0	Port 443 to App-Tier-SG; Port 80 to App-Tier-SG
App-Tier-SG	Port 443 from Web-Tier-SG; Port 443 from API Gateway	Port 443 to Bedrock endpoints; Port 443 to S3 endpoints; Port 443 to EventBridge
Data-Tier-SG	Port 443 from App-Tier-SG only	Restricted to AWS service endpoints

Network Access Control Lists (NACLs)

NACL	Inbound Rules	Outbound Rules
Public-Subnet-NACL	Rule 100: Allow HTTPS (443) from 0.0.0.0/0; Rule 110: Allow HTTP (80) from 0.0.0.0/0; Rule 120: Allow return traffic (1024-65535)	Rule 100: Allow HTTPS (443) to 0.0.0.0/0; Rule 110: Allow HTTP (80) to 0.0.0.0/0
Private-Subnet-NACL	Rule 100: Allow HTTPS (443) from VPC CIDR; Rule 110: Allow return traffic (1024-65535)	Rule 100: Allow HTTPS (443) to 0.0.0.0/0; Rule 110: Allow AWS service endpoints

Data Flow Security

Document Processing Security Flow

Stage	Flow
1. Client Upload	Internet → CloudFront → WAF → ALB → API Gateway
2. Document Processing	API Gateway → Lambda (Private) → S3 (Encrypted) → Bedrock
3. Data Storage	Bedrock → EventBridge → Lambda → S3 (Encrypted Storage)
4. Response Flow	Lambda → API Gateway → CloudFront → Client

Encryption Boundaries

Type	Components	Configuration
Data at Rest	S3 Buckets	KMS encryption (Customer Managed Keys); Bucket policies with least privilege
Data at Rest	Lambda Environment Variables	KMS encryption for sensitive data
Data in Transit	External Communications	TLS 1.2+ for all client connections; Certificate pinning for mobile apps

Data in Transit	Internal Communications	TLS 1.2+ between all AWS services; VPC endpoints for AWS service communication
-----------------	-------------------------	--

Identity and Access Management Boundaries

IAM Role Structure

Role	Permissions
LoanProcessingLambdaRole	bedrock:InvokeModel (specific models only); s3:GetObject, s3:PutObject (specific buckets); events:PutEvents (EventBridge); logs:CreateLogGroup, logs:CreateLogStream
BedrockServiceRole	s3:GetObject (document buckets only); Limited to specific Bedrock models
CognitoUserRole	execute-api:Invoke (specific API Gateway endpoints); Limited session duration (1 hour)

Monitoring and Logging Boundaries

Security Monitoring

Component	Configuration
VPC Flow Logs	Capture all network traffic; Store in CloudWatch Logs; Retention: 1 year minimum
CloudTrail	All API calls logged; Multi-region trail; Log file integrity validation
GuardDuty	Threat detection enabled; Malware protection for S3; Runtime monitoring for Lambda

Compliance Boundaries

PCI DSS Compliance Zones

Zone	Classification	Components
Zone 1	Public (Out of Scope)	CloudFront; Public Load Balancers; WAF
Zone 2	Application (In Scope)	API Gateway; Lambda Functions; Cognito; Application Logic
Zone 3	Data (Highly Restricted)	S3 Document Storage; Bedrock AI Processing; Sensitive Data Processing; Audit Logs

Network Architecture Diagram Structure

Layer	Component
-------	-----------

External	Internet Gateway
Edge	CloudFront + WAF
Public	Application Load Balancer (Public Subnets)
API	API Gateway (Private Subnets)
Compute	Lambda Functions (Private Subnets)
Data	Bedrock + S3 + EventBridge (Data Subnets)
Connectivity	VPC Endpoints (Private connectivity to AWS services)

Critical Component Details Issues & Recommendations

1. Lambda Function Specifications (ARCH-007)

Current Gap: Missing detailed Lambda function specifications (memory, timeout, concurrency)
Specific Recommendations:

Document Processing Lambda

- **Memory:** 3008 MB (for large PDF processing with Bedrock integration)
- **Timeout:** 15 minutes (for complex document analysis)
- **Concurrency:** Reserved 100, Provisioned 20 (to handle peak loads)
- **Runtime:** Python 3.11 or Node.js 18.x
- **Environment Variables:** Bedrock model ARN, S3 bucket names, encryption keys
- **VPC Configuration:** Deploy in private subnets with NAT Gateway access

API Gateway Lambda Functions

- **Memory:** 512 MB (for API request handling)
- **Timeout:** 30 seconds (for API responses)
- **Concurrency:** Reserved 200, Provisioned 50
- **Runtime:** Python 3.11 for consistency
- **Layers:** Shared libraries for AWS SDK, validation, logging

Workflow Orchestration Lambda

- **Memory:** 1024 MB (for Step Functions coordination)
- **Timeout:** 5 minutes (for workflow management)
- **Concurrency:** Reserved 50, Provisioned 10
- **Dead Letter Queue:** SQS queue for failed executions

2. S3 Bucket Configurations & Lifecycle Policies

Current Gap: Missing S3 bucket configurations, lifecycle policies, and access patterns

Specific Recommendations:

Document Storage Bucket

```
{
  "BucketName": "mortgage-docs-{environment}-{region}",
  "Versioning": "Enabled",
```

```

"Encryption": {
  "Type": "SSE-KMS",
  "KMSKeyId": "arn:aws:kms:region:account:key/key-id"
},
"LifecyclePolicy": {
  "Rules": [
    {
      "Status": "Enabled",
      "Transitions": [
        {
          "Days": 30,
          "StorageClass": "STANDARD_IA"
        },
        {
          "Days": 90,
          "StorageClass": "GLACIER"
        },
        {
          "Days": 2555,
          "StorageClass": "DEEP_ARCHIVE"
        }
      ]
    }
  ]
},
"PublicAccessBlock": "All",
"NotificationConfiguration": {
  "LambdaConfigurations": [
    {
      "Event": "s3:ObjectCreated:*",
      "LambdaFunctionArn": "document-processor-lambda-arn"
    }
  ]
}
}

```

Processed Results Bucket

- **Purpose:** Store processed mortgage application results
- **Retention:** 7 years (regulatory requirement)
- **Access Pattern:** Frequent access first 30 days, then archival
- **Cross-Region Replication:** Enable for disaster recovery

3. Amazon Bedrock Agent Configurations

Current Gap: Missing Bedrock agent configurations and prompt engineering details

Specific Recommendations:

Document Analysis Agent

```

{
  "AgentName": "mortgage-document-analyzer",
  "ModelId": "anthropic.claude-3-5-sonnet-20241022-v2:0",

```

```

"Instructions": "You are a mortgage document analysis specialist...",
"KnowledgeBase": {
  "DataSource": "mortgage-regulations-kb",
  "ChunkingStrategy": "FIXED_SIZE",
  "ChunkSize": 300,
  "OverlapPercentage": 20
},
"ActionGroups": [
  {
    "ActionGroupName": "document-validation",
    "Description": "Validate mortgage document completeness",
    "ApiSchema": "mortgage-validation-api-schema.json"
  }
],
"PromptOverrideConfiguration": {
  "PromptConfigurations": [
    {
      "PromptType": "PRE_PROCESSING",
      "PromptCreationMode": "OVERRIDDEN",
      "PromptState": "ENABLED",
      "BasePromptTemplate": "Analyze the mortgage document for..."
    }
  ]
}
}

```

DTI Calculation Agent

- **Model:** Claude 3.5 Sonnet v2 for complex financial calculations
- **Memory Configuration:** 30-day conversation memory
- **Guardrails:** Implement financial calculation validation rules
- **Action Groups:** Income verification, debt calculation, ratio analysis

4. API Gateway Specifications & Rate Limiting

Current Gap: Missing API Gateway specifications and rate limiting

Specific Recommendations:

REST API Configuration

```

{
  "ApiName": "mortgage-processing-api",
  "EndpointType": "REGIONAL",
  "Authentication": {
    "Type": "COGNITO_USER_POOLS",
    "AuthorizerId": "mortgage-app-authorizer"
  },
  "Throttling": {
    "BurstLimit": 2000,
    "RateLimit": 1000
  },
  "Caching": {
    "Enabled": true,

```

```

    "TTL": 300,
    "CacheKeyParameters": ["userId", "applicationId"]
  },
  "RequestValidation": {
    "ValidateRequestBody": true,
    "ValidateRequestParameters": true
  }
}

```

Endpoint Specifications

- **POST /applications:** Create new mortgage application
 - Rate Limit: 10 requests/minute per user
 - Payload Size: Max 50MB (for document uploads)
 - Timeout: 29 seconds
- **GET /applications/{id}:** Retrieve application status
 - Rate Limit: 100 requests/minute per user
 - Cache TTL: 60 seconds
- **POST /documents/upload:** Upload mortgage documents
 - Rate Limit: 5 requests/minute per user
 - Payload Size: Max 10MB per document

5. DynamoDB Table Specifications

Current Gap: Missing database specifications and access patterns

Specific Recommendations:

Applications Table

```

{
  "TableName": "MortgageApplications",
  "BillingMode": "ON_DEMAND",
  "AttributeDefinitions": [
    {
      "AttributeName": "applicationId",
      "AttributeType": "S"
    },
    {
      "AttributeName": "userId",
      "AttributeType": "S"
    },
    {
      "AttributeName": "status",
      "AttributeType": "S"
    }
  ],
  "KeySchema": [
    {
      "AttributeName": "applicationId",
      "KeyType": "HASH"
    }
  ],
  "GlobalSecondaryIndexes": [

```

```

{
  "IndexName": "UserApplicationsIndex",
  "KeySchema": [
    {
      "AttributeName": "userId",
      "KeyType": "HASH"
    }
  ]
},
{
  "PointInTimeRecoverySpecification": {
    "PointInTimeRecoveryEnabled": true
  },
  "SSESpecification": {
    "SSEEnabled": true,
    "KMSMasterKeyId": "alias/dynamodb-key"
  }
}

```

6. CloudFront Distribution Configuration

Current Gap: Missing CDN specifications and caching strategies

Specific Recommendations:

Distribution Configuration

```

{
  "DistributionConfig": {
    "Origins": [
      {
        "Id": "S3-mortgage-frontend",
        "DomainName": "mortgage-frontend-bucket.s3.amazonaws.com",
        "S3OriginConfig": {
          "OriginAccessIdentity": "origin-access-identity/cloudfront/ABCDEFGF1234567"
        }
      },
      {
        "Id": "API-Gateway-mortgage-api",
        "DomainName": "api.mortgage-app.com",
        "CustomOriginConfig": {
          "HTTPPort": 443,
          "OriginProtocolPolicy": "https-only"
        }
      }
    ],
    "DefaultCacheBehavior": {
      "TargetOriginId": "S3-mortgage-frontend",
      "ViewerProtocolPolicy": "redirect-to-https",
      "CachePolicyId": "managed-caching-optimized",
      "TTL": {
        "DefaultTTL": 86400,
        "MaxTTL": 31536000
      }
    }
  }
}

```

```

    }
  },
  "CacheBehaviors": [
    {
      "PathPattern": "/api/*",
      "TargetOriginId": "API-Gateway-mortgage-api",
      "CachePolicyId": "managed-caching-disabled",
      "OriginRequestPolicyId": "managed-cors-s3-origin"
    }
  ]
}
}

```

7. Amazon Cognito User Pool Configuration

Current Gap: Missing authentication and authorization specifications

Specific Recommendations:

User Pool Configuration

```

{
  "UserPoolName": "mortgage-app-users",
  "Policies": {
    "PasswordPolicy": {
      "MinimumLength": 12,
      "RequireUppercase": true,
      "RequireLowercase": true,
      "RequireNumbers": true,
      "RequireSymbols": true,
      "TemporaryPasswordValidityDays": 7
    }
  },
  "MfaConfiguration": "OPTIONAL",
  "EnabledMfas": ["SMS_MFA", "SOFTWARE_TOKEN_MFA"],
  "AccountRecoverySetting": {
    "RecoveryMechanisms": [
      {
        "Name": "verified_email",
        "Priority": 1
      }
    ]
  },
  "UserPoolAddOns": {
    "AdvancedSecurityMode": "ENFORCED"
  },
  "Schema": [
    {
      "Name": "email",
      "AttributeDataType": "String",
      "Required": true,
      "Mutable": true
    },
  ]
}

```

```
"Name": "phone_number",
"AttributeDataType": "String",
"Required": false,
"Mutable": true
}
]
```

Integration Specifications Between Components

1. Lambda to Bedrock Integration

- **SDK Version:** AWS SDK v3 for optimal performance
- **Connection Pooling:** Reuse HTTP connections
- **Error Handling:** Exponential backoff with jitter
- **Timeout Configuration:** 60 seconds for Bedrock calls
- **Retry Logic:** 3 attempts with circuit breaker pattern

2. API Gateway to Lambda Integration

- **Integration Type:** Lambda Proxy Integration
- **Request Mapping:** Pass through all request parameters
- **Response Mapping:** Standard HTTP status codes
- **Error Handling:** Custom error responses with correlation IDs

3. S3 to Lambda Event Integration

- **Event Type:** ObjectCreated events
- **Filter Rules:** Only process PDF, DOCX, and image files
- **Batch Size:** Process one document at a time
- **Error Handling:** Dead letter queue for failed processing

Resource Sizing & Capacity Planning

Compute Resources

- **Total Lambda Memory:** 15 GB across all functions
- **API Gateway RPS:** 1,000 requests per second sustained
- **Bedrock Invocations:** 500 per minute with burst to 1,000

Storage Resources

- **S3 Storage:** 1 TB initial capacity, 100 GB growth per month
- **DynamoDB:** 1,000 RCU/WCU with auto-scaling enabled
- **CloudWatch Logs:** 30-day retention, 10 GB per month

1. Disaster Recovery & Business Continuity Planning (ARCH-003)

Current Gap: No disaster recovery or business continuity planning documented

Specific Recommendations:

- Implement Multi-Region Architecture: Deploy critical components across at least 2 AWS regions (primary and DR)
- Define RTO/RPO Objectives:
 - RTO (Recovery Time Objective): Target 4 hours for mortgage processing systems
 - RPO (Recovery Point Objective): Target 15 minutes maximum data loss
- Create DR Runbooks: Document step-by-step recovery procedures for each component
- Automated Backup Strategy:
 - S3 Cross-Region Replication for document storage
 - DynamoDB Global Tables for application data
 - Lambda function versioning and automated deployment
- Regular DR Testing: Quarterly disaster recovery drills with documented results

Fault Tolerance Patterns (ARCH-005)

Current Gap: Missing fault tolerance patterns and error handling strategies

Specific Recommendations:

- Circuit Breaker Pattern: Implement for Bedrock API calls to prevent cascade failures
- Retry Logic with Exponential Backoff: For all external service integrations
- Dead Letter Queues: For failed document processing workflows
- Health Checks: Implement comprehensive health monitoring for all components
- Graceful Degradation: Design fallback mechanisms when AI services are unavailable

3. Multi-AZ Deployment Architecture

Current Gap: No multi-AZ deployment documented

Specific Recommendations:

- Application Load Balancer: Deploy across multiple AZs with health checks
- Lambda Functions: Ensure deployment across multiple AZs automatically
- Database Layer: Use RDS Multi-AZ or DynamoDB with Global Tables
- S3 Storage: Enable Cross-Region Replication for critical documents
- Network Design: Implement subnets across 3+ AZs in each region

4. Monitoring & Alerting Framework

Current Gap: Insufficient monitoring and alerting specifications

Specific Recommendations:

- CloudWatch Dashboards: Create real-time visibility into system health
- Custom Metrics: Track mortgage processing SLAs and business KPIs
- Automated Alerting: Set up alerts for:
 - Processing time > 30 minutes
 - Error rate > 1%
 - Document processing failures
 - Bedrock API throttling

- Log Aggregation: Centralized logging with CloudWatch Logs and X-Ray tracing
- Incident Response: 24/7 on-call rotation with escalation procedures

5. Backup & Recovery Strategies

Current Gap: No backup strategies documented

Specific Recommendations:

- Automated Daily Backups: For all data stores with 30-day retention
- Point-in-Time Recovery: Enable for critical databases
- Document Versioning: Implement S3 versioning for all mortgage documents
- Configuration Backup: Version control for all infrastructure as code
- Recovery Testing: Monthly backup restoration tests with validation

6. Error Handling & Data Integrity

Current Gap: Missing comprehensive error handling

Specific Recommendations:

- Transaction Management: Implement ACID properties for mortgage data
- Data Validation: Multi-layer validation for all document processing
- Audit Trails: Complete audit logging for all mortgage processing steps
- Rollback Mechanisms: Ability to reverse processing steps if errors detected
- Data Consistency Checks: Regular validation of data integrity across systems

Implementation Priority (30-Day Action Plan)

Week 1-2: Foundation

1. Implement basic monitoring and alerting
2. Set up automated backups for all data stores
3. Document RTO/RPO objectives and initial DR procedures

Week 3-4: Fault Tolerance

1. Implement circuit breaker patterns for Bedrock integration
2. Add retry logic and dead letter queues
3. Deploy multi-AZ architecture components

Month 2: Advanced Reliability

1. Complete multi-region DR setup
2. Conduct first disaster recovery test
3. Implement comprehensive audit trails

Success Metrics

- System Availability: Target 99.9% uptime (8.76 hours downtime/year)
- Mean Time to Recovery (MTTR): < 30 minutes for non-critical issues

- Data Loss Prevention: Zero data loss during planned maintenance
- Processing SLA: 95% of mortgage applications processed within defined timeframes

Performance Testing & Benchmarking (ARCH-004)

Current Gap: No performance testing or benchmarking documented

Specific Recommendations:

- Load Testing Framework: Implement comprehensive load testing using AWS Load Testing Solution
- Performance Baselines: Establish baseline metrics for:
 - Document processing time: Target < 30 seconds per document
 - API response times: Target < 2 seconds for all endpoints
 - Concurrent user capacity: Support 1,000+ simultaneous mortgage applications
- Stress Testing: Test system behavior under 3x expected peak load
- Endurance Testing: 24-hour continuous load testing to identify memory leaks
- Performance Test Automation: Integrate performance tests into CI/CD pipeline

2. Capacity Planning & Scaling Strategies (ARCH-008)

Current Gap: Missing capacity planning and scaling strategies

Specific Recommendations:

- Auto-Scaling Policies: Configure intelligent scaling for:
 - Lambda concurrency limits based on processing queue depth
 - API Gateway throttling based on usage patterns
 - Bedrock model invocation limits with burst capacity
- Capacity Models: Develop predictive models for:
 - Peak mortgage application periods (month-end, seasonal trends)
 - Document processing volume forecasting
 - Storage growth projections
- Resource Right-Sizing: Optimize Lambda memory allocation based on actual usage patterns
- Reserved Capacity: Pre-provision capacity for predictable workloads

3. Performance Monitoring & Optimization

Current Gap: No performance monitoring or optimization strategies

Specific Recommendations:

- Real-Time Performance Dashboards: Create CloudWatch dashboards tracking:
 - End-to-end processing latency
 - Bedrock API response times and throttling
 - Lambda cold start frequency and duration
 - S3 upload/download performance

- Performance Alerting: Set up alerts for:
 - Processing time > 45 seconds (warning), > 60 seconds (critical)
 - API latency > 3 seconds
 - Lambda timeout rate > 0.1%
 - Bedrock throttling events
- Performance Analytics: Implement detailed performance tracking with:
 - X-Ray distributed tracing for end-to-end visibility
 - Custom CloudWatch metrics for business KPIs
 - Performance trend analysis and reporting

4. Lambda Function Optimization

Current Gap: No Lambda performance optimization documented

Specific Recommendations:

- Memory Optimization: Right-size Lambda functions based on CPU/memory profiling
- Cold Start Reduction:
 - Implement provisioned concurrency for critical functions
 - Optimize package size and dependencies
 - Use Lambda layers for shared libraries
- Connection Pooling: Implement connection reuse for database and external API calls
- Async Processing: Use SQS/SNS for non-blocking document processing workflows
- Function Splitting: Break large functions into smaller, focused microservices

5. Bedrock AI/ML Performance Optimization

Current Gap: No AI service performance optimization

Specific Recommendations:

- Model Selection Optimization:
 - Use Claude Sonnet 3.5 v2 for complex document analysis
 - Consider Claude Haiku for simple classification tasks (faster, cheaper)
- Prompt Engineering: Optimize prompts for:
 - Reduced token usage (cost and latency optimization)
 - Consistent output format to minimize parsing overhead
- Batch Processing: Group similar documents for batch processing when possible
- Caching Strategy: Implement intelligent caching for:
 - Frequently processed document types
 - Common mortgage calculation results
- Request Optimization: Minimize API calls through efficient prompt design

6. Data Access & Storage Performance

Current Gap: No storage performance optimization

Specific Recommendations:

- S3 Performance Optimization:
 - Use S3 Transfer Acceleration for large document uploads
 - Implement multipart uploads for files > 100MB
 - Optimize S3 request patterns to avoid hot-spotting

- Database Performance:
 - Implement DynamoDB with appropriate partition key design
 - Use DynamoDB Accelerator (DAX) for microsecond latency
 - Optimize query patterns and indexes
- Content Delivery: Use CloudFront for static assets and frequently accessed documents
- Data Compression: Implement compression for document storage and transfer

7. Network & API Performance

Current Gap: Missing network optimization strategies

Specific Recommendations:

- API Gateway Optimization:
 - Enable response caching for static responses
 - Implement request/response compression
 - Use regional endpoints for better latency
- VPC Optimization: Design VPC with optimal routing and minimal network hops
- CDN Strategy: Implement CloudFront for global content delivery
- Connection Optimization: Use HTTP/2 and connection keep-alive

Performance Targets & SLAs

Processing Performance

- Document Upload: < 5 seconds for files up to 10MB
- Document Analysis: < 30 seconds per document (average)
- Mortgage Calculation: < 2 seconds for DTI and pre-approval
- End-to-End Processing: < 60 seconds for complete application

System Performance

- API Response Time: 95th percentile < 2 seconds
- System Availability: 99.9% uptime during business hours
- Concurrent Users: Support 1,000+ simultaneous users
- Throughput: Process 10,000+ documents per hour during peak

Cost Efficiency Targets

- Lambda Cost Optimization: Reduce execution time by 30% through optimization
- Bedrock Cost Reduction: Optimize token usage to reduce AI costs by 25%
- Storage Efficiency: Implement compression to reduce storage costs by 20%

Monitoring & Alerting Strategy

Key Performance Indicators (KPIs)

- Business Metrics: Application processing time, approval rate, user satisfaction
- Technical Metrics: Latency, throughput, error rates, resource utilization
- Cost Metrics: Per-transaction cost, resource efficiency, optimization savings

Alert Thresholds

- Critical: Processing time > 60 seconds, Error rate > 1%
- Warning: Processing time > 45 seconds, Error rate > 0.5%
- Info: Unusual traffic patterns, capacity threshold approaching

Cost Estimation

- Approximate cost: The Auto Loan Application will cost \$226 per month for 1,000 pages, 28,800 requests (us-west-2 region, April 2025)
- Recommend setting up [AWS Budget](#)

Cost Breakdown

Below you can see the estimated costs of the guidance solution and the assumptions.

AWS service	Dimensions	Cost [USD]
Amazon Bedrock Data Automation	1,000 documents (each 1 page) processed with Bedrock Data Automation	\$40
Amazon Bedrock Agent - Claude Sonnet 3.5 v2	Average 2 requests to the Bedrock agent per minutes, input tokens 1000 per request, output tokens 200 per request	\$173
AWS Lambda	28,800 requests per month on Arm with 512 MB ephemeral storage, 30s execution time	\$0.32
Amazon EventBridge	1 million events	\$1
Amazon Simple Storage Service (S3)	S3 Standard storage (10 GB per month)	\$0.24
Amazon CloudFront	Data transfer out to internet cost: \$0.85 and Data transfer out to origin cost: \$0.20 (10 GB per month), 28,000 requests cost: \$0.03	\$1.08
Amazon Cognito	100 monthly active users (MAU) with Advanced security feature (WAF)	\$10.75